**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

# ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №1. Алгоритмы сортировки в оперативной памяти

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151438 _____ Д.А, Пузынин
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

## ЦЕЛЬ РАБОТЫ

Изучение алгоритмов сортировки массивов данных в ОП, особенностей их программной реализации и эффективности работы на различных наборах исходных данных.

## ПОСТАНОВКА ЗАДАЧИ

Разработать и реализовать программу, предназначенную для исследования времени работы и поведения методов сортировки, указанных в варианте задания.

Для решения поставленной задачи применить объектно-ориентированный подход. Разработать шаблон класса «Одномерный массив», включающий в себя необходимый минимум методов, обеспечивающий полноценное функционирование объектов указанного класса в рамках данной программы: конструкторы (по умолчанию, с параметрами, копирования), деструктор, методы ввода данных в массив с клавиатуры, заполнения случайными значениями из заданного диапазона, вывода массива на экран, вывода массива в файл, сортировки алгоритмами, указанными в варианте задания.

Провести исследование параметров работы указанных в варианте задания методов сортировки на различных наборах данных. Представить отчет, содержащий результаты исследования и полученные выводы.

**Вариант №13**

Программно реализовать следующие методы сортировки данных в оперативной памяти: сортировка методом Шелла; пирамидальная сортировка.

Оценить быстродействие указанных методов и степень естественности их поведения.

## ХОД РАБОТЫ

В ходе лабораторной работы была разработана программа, предназначенная для исследования времени работы и поведения методов сортировки Шелла и пирамидальной. С помощью ООП был разработан шаблон класса Одномерный массив (Рисунок 1). Были реализованы конструкторы копирования, по умолчанию, с параметрами, оператор присваивания, деструктор (Рисунок 2).

```
template<typename T>
class Array {
private:
    T* data;
    int size;
```

*Рисунок 1 – одномерный массив*

```
public:
    Array() : data(nullptr), size(0) {}

    Array(int size) : size(size) {
        if (size < 0) throw std::invalid_argument("Размер списка не может быть отрицательным");
        data = new T[size];
    }

    Array(const Array& other) : size(other.size) {
        data = new T[size];
        for (int i = 0; i < size; ++i) {
            data[i] = other.data[i];
        }
    }

    Array& operator=(const Array& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new T[size];
            for (int i = 0; i < size; ++i) {
                data[i] = other.data[i];
            }
        }
        return *this;
    }

    ~Array() {
        delete[] data;
    }
```

*Рисунок 2 – Конструкторы*

Были реализованы методы ввода массива с клавиатуры и случайное заполнение массива, заполнение упорядоченными значениями, значениями в обратном порядке (рис. 3-6)

```
void inputFromKeyboard() {
    std::cout << "Введите " << size << " элементов массива:\n";
    for (int i = 0; i < size; ++i) {
        std::cout << "Элемент [" << i << "]: ";
        std::cin >> data[i];
    }
}
```

*Рисунок 3 - Заполнение массива с клавиатуры*

```
void fillRandom(T min_val, T max_val) {
    std::random_device rd;
    std::mt19937 gen(rd());
```

*Рисунок 4 - Случайное заполнение массива*

```
void fillSorted() {
    for (int i = 0; i < size; ++i) {
        data[i] = static_cast<T>(i);
    }
}
```

*Рисунок 5 - Заполнение упорядоченными значениями*

```
void fillReverseSorted() {
    for (int i = 0; i < size; ++i) {
        data[i] = static_cast<T>(size - 1 - i);
    }
}
```

*Рисунок 6 - Заполнение значениями в обратном порядке*

Был реализован метод вывода массива на экран (Рисунок 7):

```
void display() const {
    std::cout << "Массив (первые 20 элементов): ";
    for (int i = 0; i < std::min(size, 20); ++i) {
        std::cout << data[i] << " ";
    }
    if (size > 20) {
        std::cout << "...";
    }
    std::cout << std::endl;
}
```

*Рисунок 7 - Метод вывода массива на экран*

Метод вывода массива в файл (Рисунок 8). Принимает имя файла и уже отсортированный массив для вывода обеих версий.

```
void writeToFile(const std::string& filename, const Array<T>& sorted_arr) const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Ошибка: не удалось открыть файл " << filename << std::endl;
        return;
    }

    file << "Исходный массив:\n";
    for (int i = 0; i < this->size; ++i) {
        file << this->data[i] << " ";
    }

    file << "\n\nОтсортированный массив:\n";
    for (int i = 0; i < sorted_arr.size; ++i) {
        file << sorted_arr.data[i] << " ";
    }

    file.close();
    std::cout << "Массивы успешно записаны в файл " << filename << std::endl;
}
```

*Рисунок 8 - Метод вывода массива в файл*

Сортировки методом Шелла и пирамидальная (Рисунок 9-10):

```
SortStatistics shellSort() {
    SortStatistics stats;
    for (int gap = size / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < size; ++i) {
            T temp = data[i];
            int j;
            for (j = i; j >= gap && ++stats.comparisons && data[j - gap] > temp; j -= gap) {
                data[j] = data[j - gap];
                stats.swaps++;
            }
            data[j] = temp;
            if (j != i) {
                stats.swaps++;
            }
        }
    }
    return stats;
}
```

*Рисунок 9 - Сортировка методом Шелла*

```
SortStatistics heapSort() {
    SortStatistics stats;

    for (int i = size / 2 - 1; i >= 0; --i) {
        heapify(size, i, stats);
    }

    for (int i = size - 1; i > 0; --i) {
        std::swap(data[0], data[i]);
        stats.swaps++;
        heapify(i, 0, stats);
    }
    return stats;
}
```

*Рисунок 10 - Сортировка пирамидальным методом*

Было реализовано меню, позволяющее выбирать режимы работы программы (Рисунок 11-12):

```
int main() {

    int choice;
    do {
        std::cout << "\nМеню программы:\n";
        std::cout << "1. Сортировка одного массива\n";
        std::cout << "2. Режим накопления статистики\n";
        std::cout << "3. Выход\n";
        std::cout << "Ваш выбор: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                singleSortMode();
                break;
            case 2:
                statisticsMode();
                break;
            case 3:
                std::cout << "Завершение работы программы.\n";
                break;
            default:
                std::cout << "Некорректный ввод. Пожалуйста, выберите пункт меню.\n";
        }
    } while (choice != 3);

    return 0;
}
```

*Рисунок 11 – Код меню*

```
Меню программы:
1. Сортировка одного массива
2. Режим накопления статистики
3. Выход
Ваш выбор:
```

*Рисунок 12 – Меню*

Режим сортировки одного массива (Рисунок 13) реализован следующим образом: пользователем вводится размер массива, диапазон значений (от наименьшего возможного, до наибольшего возможного), метод сортировки (в нашем случае 1 – Шелла, 2 – Пирамидальный) (Рисунок 14). Далее создаётся объект класса Array и копия для сохранения исходного состояния с помощью конструктора копирования. Далее, в зависимости от выбора пользователя, выбирается метод сортировки. После того, как сортировка будет произведена, программа выдаст статистику и предложит сохранить результат в файл (Рисунок 15).

```
void singleSortMode() {
    int size;
    std::cout << "\n--- Режим 1: Сортировка одного массива ---\n";
    std::cout << "Введите размер массива: ";
    std::cin >> size;

    int min_val, max_val;
    std::cout << "Введите диапазон значений (min max): ";
    std::cin >> min_val >> max_val;

    int method;
    std::cout << "Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): ";
    std::cin >> method;

    Array<int> arr(size);
    arr.fillRandom(min_val, max_val);

    Array<int> original_arr = arr;

    std::cout << "\nИсходный массив:\n";
    original_arr.display();

    SortStatistics stats;
    std::string methodName;

    if (method == 1) {
        methodName = "Сортировка Шелла";
        std::cout << "\nСортировка методом Шелла...\n";
        stats = arr.shellSort();
    } else if (method == 2) {
        methodName = "Пирамидальная сортировка";
        std::cout << "\nПирамидальная сортировка...\n";
        stats = arr.heapSort();
    } else {
        std::cout << "Неверный метод сортировки.\n";
        return;
    }

    std::cout << "\nСортировка завершена.\n";
    std::cout << "Отсортированный массив:\n";
    arr.display();

    std::cout << "\n--- Статистика ---\n";
    std::cout << "Метод: " << methodName << "\n";
    std::cout << "Сравнений: " << stats.comparisons << "\n";
    std::cout << "Перестановок: " << stats.swaps << "\n";

    std::string filename;
    std::cout << "\nВведите имя файла для сохранения результатов (например, result.txt): ";
    std::cin >> filename;

    original_arr.writeToFile(filename, arr);
}
```

*Рисунок 13 - Режим сортировки одного массива*

```
Меню программы:
1. Сортировка одного массива
2. Режим накопления статистики
3. Выход
Ваш выбор: 1

--- Режим 1: Сортировка одного массива ---
Введите размер массива: 10000
Введите диапазон значений (min max): 1 1000000
Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): 1

Исходный массив:
Массив (первые 20 элементов): 346380 57524 233857 102941 53801 83476 466394 895848 35436 586550 974648 614668 87573 67268 360510 561047 73546 517456 6414 613110 ...

Сортировка методом Шелла...

Сортировка завершена.
Отсортированный массив:
Массив (первые 20 элементов): 31 135 214 378 382 405 478 506 512 525 648 662 852 1020 1104 1224 1263 1309 1375 1376 ...

--- Статистика ---
Метод: Сортировка Шелла
Сравнений: 262192
Перестановок: 205369

Введите имя файла для сохранения результатов (например, result.txt): 1shelltest.txt
Массивы успешно записаны в файл 1shelltest.txt
```

*Рисунок 14 - Метод 1. Ввод данных пользователем*

```
C_codespace > 2 kurs > 1-st senester > Algorithms and data structure > 1-st lab > Dima > 1shelltest.txt
1  Исходный массив:
2  346380 57524 233857 102941 53801 83476 466394 895848 35436 586550 974648 614668 87573 67268
3
4  Отсортированный массив:
5  31 135 214 378 382 405 478 506 512 525 648 662 852 1020 1104 1224 1263 1309 1375 1376 1408 1
```

*Рисунок 15 - Файл с итогом работы 1 метода*

Режим накопления статистических данных (Рисунок 16). При выборе данного пункта у пользователя появляется выбор способов формирования элементов массива: 1. Случайные значения, 2. Упорядоченная последовательность, 3. Обратно упорядоченная последовательность. Дальнейший пример работы программы будет показан, используя упорядоченную последовательность для формирования элементов массива. Сортировка - пирамидальная. Пользователю необходимо ввести начальный размер массива, конечный размер массива и шаг изменения размера, после чего будет предложен метод сортировки (Рисунок 17). Далее программа начинает создавать массивы и сортировать их, отображая размер и время на сортировку (Рисунок 18). При завершении работы будет предложено сохранить результат в файл (Рисунок 19).

```
void statisticsMode() {
    int startSize, endSize, step;
    std::cout << "\n--- Режим 2: Сбор статистики ---\n";
    std::cout << "Введите начальный размер массива: ";
    std::cin >> startSize;
    std::cout << "Введите конечный размер массива: ";
    std::cin >> endSize;
    std::cout << "Введите шаг изменения размера: ";
    std::cin >> step;

    int arrayType;
    std::cout << "Выберите способ формирования массива:\n";
    std::cout << "1. Случайные значения\n";
    std::cout << "2. Упорядоченная последовательность\n";
    std::cout << "3. Обратно упорядоченная последовательность\n";
    std::cout << "Ваш выбор: ";
    std::cin >> arrayType;

    int method;
    std::cout << "Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): ";
    std::cin >> method;

    std::string filename;
    std::cout << "\nВведите имя файла для сохранения статистики (например, stats.csv): ";
    std::cin >> filename;

    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Ошибка: не удалось открыть файл " << filename << std::endl;
        return;
    }

    file << "ArraySize,SortTime(ms)\n";
    std::cout << "\nИдет сбор статистики...\n";

    for (int currentSize = startSize; currentSize <= endSize; currentSize += step) {
        Array<int> arr(currentSize);

        switch (arrayType) {
            case 1: arr.fillRandom(0, currentSize * 10); break;
            case 2: arr.fillSorted(); break;
            case 3: arr.fillReverseSorted(); break;
            default: arr.fillRandom(0, currentSize * 10); break;
        }

        auto startTime = std::chrono::high_resolution_clock::now();

        if (method == 1) {
            arr.shellSort();
        } else {
            arr.heapSort();
        }

        auto endTime = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> duration = endTime - startTime;

        file << currentSize << "," << std::fixed << std::setprecision(4) << duration.count() << "\n";
        std::cout << "Размер: " << currentSize << ", Время: " << std::fixed << std::setprecision(4) << duration.count() << " мс\n";
    }
```

*Рисунок 16 - Режим накопления статистических данных*

```
Меню программы:
1. Сортировка одного массива
2. Режим накопления статистики
3. Выход
Ваш выбор: 2

--- Режим 2: Сбор статистики ---
Введите начальный размер массива: 1000
Введите конечный размер массива: 50000
Введите шаг изменения размера: 1000
Выберите способ формирования массива:
1. Случайные значения
2. Упорядоченная последовательность
3. Обратно упорядоченная последовательность
Ваш выбор: 2
Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): 2
```

*Рисунок 17 - Метод 2. Ввод данных пользователем*

```
Идет сбор статистики...
Размер: 1000, Время: 0.1408 мс
Размер: 2000, Время: 0.2436 мс
Размер: 3000, Время: 0.3966 мс
Размер: 4000, Время: 0.5495 мс
Размер: 5000, Время: 0.6928 мс
Размер: 6000, Время: 0.8417 мс
Размер: 7000, Время: 0.9923 мс
Размер: 8000, Время: 1.1475 мс
Размер: 9000, Время: 1.5302 мс
Размер: 10000, Время: 1.5666 мс
Размер: 11000, Время: 2.0307 мс
Размер: 12000, Время: 1.7527 мс
Размер: 13000, Время: 2.1987 мс
Размер: 14000, Время: 2.4511 мс
Размер: 15000, Время: 2.4740 мс
Размер: 16000, Время: 2.9408 мс
Размер: 17000, Время: 3.1367 мс
Размер: 18000, Время: 2.9545 мс
Размер: 19000, Время: 3.2222 мс
Размер: 20000, Время: 3.5246 мс
Размер: 21000, Время: 3.9756 мс
Размер: 22000, Время: 3.7125 мс
Размер: 23000, Время: 3.8482 мс
Размер: 24000, Время: 3.6519 мс
Размер: 25000, Время: 4.3647 мс
Размер: 26000, Время: 4.3924 мс
Размер: 27000, Время: 6.5425 мс
Размер: 28000, Время: 4.3549 мс
Размер: 29000, Время: 5.0613 мс
Размер: 30000, Время: 5.5306 мс
Размер: 31000, Время: 5.0595 мс
```

*Рисунок 18 - Сбор статистики*

C_codespace > 2 kurs > 1-st senester > Algorithms and data structure > 1-st lab > Dima > 2heaptest.csv

```
ArraySize,SortTime(ms)
1000,0.1408
2000,0.2436
3000,0.3966
4000,0.5495
5000,0.6928
6000,0.8417
7000,0.9923
8000,1.1475
9000,1.5302
10000,1.5666
11000,2.0307
12000,1.7527
13000,2.1987
14000,2.4511
15000,2.4740
16000,2.9408
17000,3.1367
18000,2.9545
19000,3.2222
20000,3.5246
21000,3.9756
22000,3.7125
23000,3.8482
24000,3.6519
25000,4.3647
26000,4.3924
27000,6.5425
28000,4.3549
```

*Рисунок 19 - Файл со статистикой*

Результат работы программы в первом режиме. Был выбран массив размером в 50000. Минимальный элемент – 1, максимальный – 100000. В сортировке Шелла вышло 1916905 сравнений и 1598970 перестановок (Рисунок 20). В пирамидальной – 1409920 сравнений и 737547 перестановок (Рисунок 21). Вычисления были проделаны ещё 4 раза для программных ошибок. Результаты отличались с максимальной погрешностью в 10%.

```
--- Режим 1: Сортировка одного массива ---
Введите размер массива: 50000
Введите диапазон значений (min max): 1 100000
Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): 1

Исходный массив:
Массив (первые 20 элементов): 10056 6117 10227 14502 15906 22543 99388 26136 7068 31639 73873 70083 41319 70680 43832 94671 96335 63123 53246 63759 ...

Сортировка методом Шелла...

Сортировка завершена.
Отсортированный массив:
Массив (первые 20 элементов): 2 4 6 9 15 17 18 19 26 28 28 29 34 37 44 48 56 57 58 62 ...

--- Статистика ---
Метод: Сортировка Шелла
Сравнений: 1916905
Перестановок: 1598970

Введите имя файла для сохранения результатов (например, result.txt): resultshell.txt
Массивы успешно записаны в файл resultshell.txt
```

*Рисунок 20 - Режим 1. Метод Шелла*

```
Меню программы:
1. Сортировка одного массива
2. Режим накопления статистики
3. Выход
Ваш выбор: 1

--- Режим 1: Сортировка одного массива ---
Введите размер массива: 50000
Введите диапазон значений (min max): 1 100000
Выберите метод сортировки (1 - Шелла, 2 - Пирамидальная): 2

Исходный массив:
Массив (первые 20 элементов): 47721 241 73243 7147 84373 83322 29806 42014 37882 1425 29990 78372 64841 42300 8550 16010 50035 99762 71417 47267 ...

Пирамидальная сортировка...

Сортировка завершена.
Отсортированный массив:
Массив (первые 20 элементов): 2 4 5 6 13 13 14 16 18 19 20 22 23 24 25 28 29 30 32 37 ...

--- Статистика ---
Метод: Пирамидальная сортировка
Сравнений: 1409920
Перестановок: 737547

Введите имя файла для сохранения результатов (например, result.txt): resultheap.txt
Массивы успешно записаны в файл resultheap.txt
```

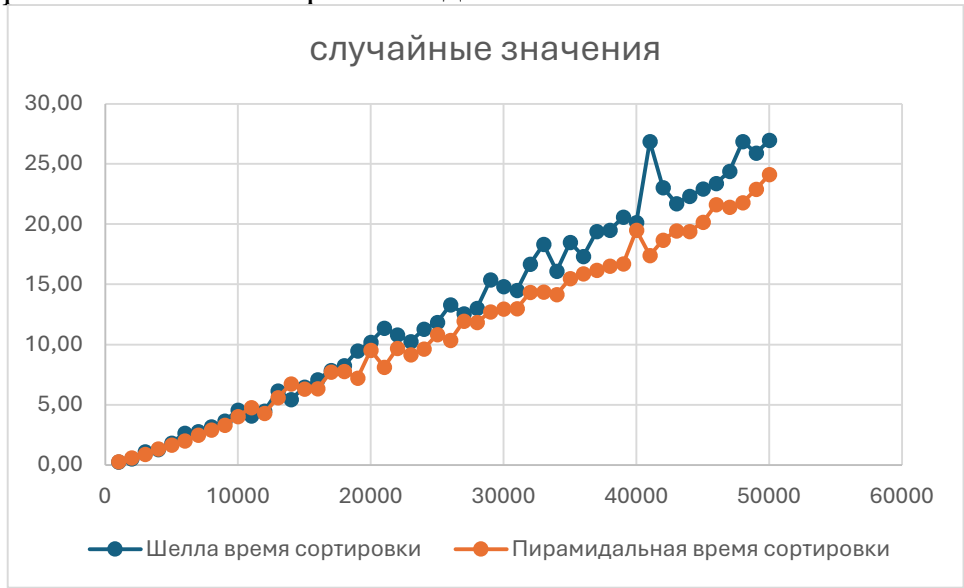
*Рисунок 21 - Режим 1. Пирамидальный метод*

Для сравнения методов сортировки были выбраны параметры:

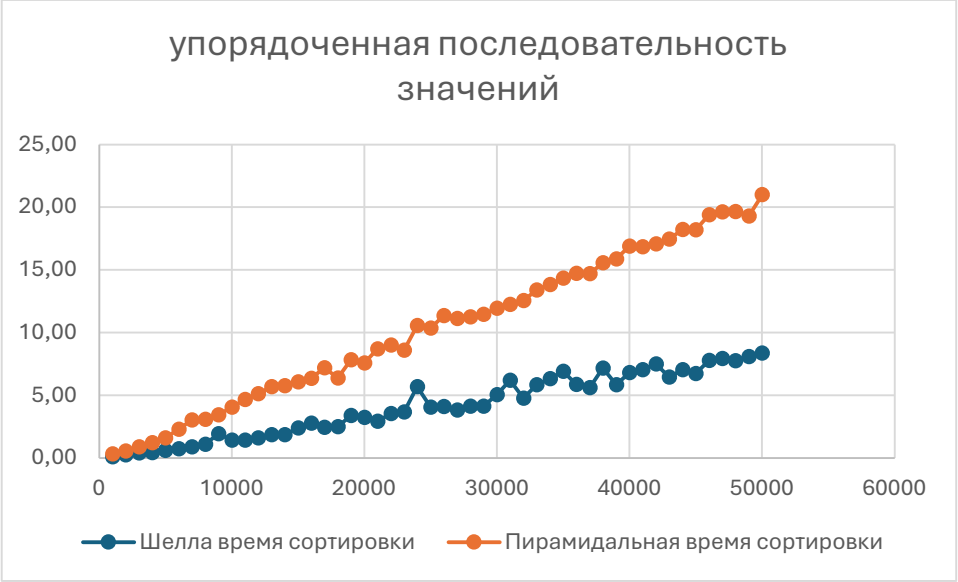
- начальный размер массива – 1000;
- конечный размер массива – 50000;
- шаг изменения массива – 1000.

При данных значениях выполнение программы не требует большого количества времени, а количество шагов отлично подходит для детального отображения на графике затраченного на сортировку времени.

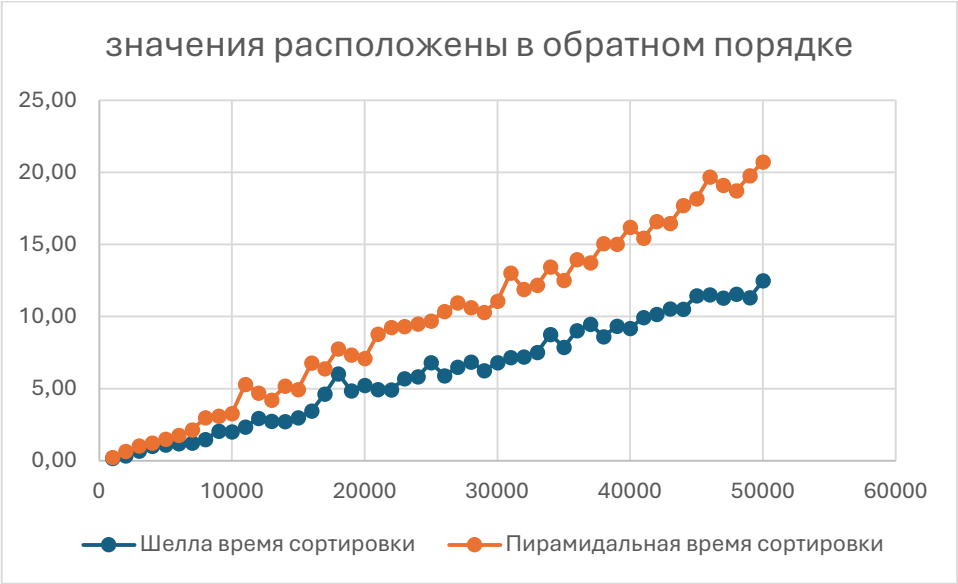
При заполнении массива случайными значениями время на сортировку методами Шелла и Пирамидальным примерно равно. Пирамидальный метод сортировки на 10% быстрее метода Шелла




Самая большая разница получилась при сортировке списков, заполненных упорядоченной последовательностью значений. Метод Шелла справляется с поставленной задачей в 2,3 раза быстрее.




В списках, заполненных значениями, расположенными в обратном порядке, метод Шелла также занимает лидирующее положение. Он быстрее Пирамидального метода в 1,6 раз быстрее.


значения расположены в обратном порядке
25,00
20,00
15,00
10,00
5,00
0,00
0
10000
20000
30000
40000
50000
60000
Шелла время сортировки
Пирамидальная время сортировки

## ВЫВОДЫ

В ходе лабораторной работы были изучены методы сортировки массивов данных, особенности их программной реализации и эффективности работы на различных наборах исходных данных.

При заполнении списков случайными значениями Пирамидальный метод сортировки показал себя лучше, чем метод Шелла. Это подтверждают разница в количестве сравнений и перестановок, и график. Но при упорядоченной последовательности и значениях в обратном порядке метод Шелла быстрее в среднем в 2 раза. Это говорит о том, что метод Шелла имеет высокую степень естественности в отличие от Пирамидального. Сумма этих факторов ставит метод Шелла на первое место.